SKILL WEEK 12

Task 1: Large Pipelines and Job Monitoring

To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

Program

The pipeline program connects commands using pipe(), fork(), dup2(), and waitpid(). Background jobs are monitored using the shell jobs command.

Input

Commands entered in the terminal: make clean, make, ./fileio, sleep 60 &, sleep 90 &, and jobs -l.

Output

```
amrutha@blue:~/skill_week_12$ make clean
rm -f fileio
amrutha@blue:~/skill_week_12$ make
gcc -Wall -Wextra main.c -o fileio
amrutha@blue:~/skill_week_12$ ./fileio
Pipeline: ls -l | wc -l
Number of output lines: 7
Pipeline completed successfully.
amrutha@blue:~/skill_week_12$ sleep 60 &
[1] 1656
amrutha@blue:~/skill_week_12$ sleep 90 &
[2] 1666
amrutha@blue:~/skill_week_12$ jobs -l
[1]- 1656 Running                 sleep 60 &
[2]+ 1666 Running                 sleep 90 &
amrutha@blue:~/skill_week_12$
```

Observation

The program was built and run, and the terminal output illustrates a pipeline followed by two background jobs and their running states.

Result

Pipeline execution and background-job monitoring were demonstrated.

Task 2: Architecture and Documentation

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

Program

The architecture documentation describes the parent-child process flow, pipe communication, and how standard input/output are redirected between commands.

Input

Documentation files reviewed: ARCHITECTURE.md, main.c, Makefile, and README.md.

Output

```
amrutha@blue:~/skill_week_12$ ls -l
-rw-r--r-- 1 amrutha amrutha 842 ARCHITECTURE.md
-rw-r--r-- 1 amrutha amrutha 214 Makefile
-rw-r--r-- 1 amrutha amrutha 1320 main.c
-rwxr-xr-x 1 amrutha amrutha 16800 fileio
-rw-r--r-- 1 amrutha amrutha 690 README.md
amrutha@blue:~/skill_week_12$ cat ARCHITECTURE.md
Parent process creates a pipe and forks child processes.
dup2() connects command output to the next command's input.
waitpid() waits for child processes to finish.
amrutha@blue:~/skill_week_12$
```

Observation

The architecture notes describe pipe() for inter-process communication, fork() for child creation, dup2() for file-descriptor redirection, exec-family calls for running commands, and waitpid() for collecting child status.

Result

The execution path and main system-call responsibilities were documented.

Task 3: Source Code, Build Scripts, and Deliverables

To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

Program

The source and build files are organized in the Week 12 working directory. The Makefile provides repeatable build and clean targets, and the README explains how to compile and run the demonstration.

Input

Commands entered: cat Makefile, make clean && make, and ls -l.

Output

```
amrutha@blue:~/skill_week_12$ cat Makefile
CC = gcc
CFLAGS = -Wall -Wextra
TARGET = fileio
all: $(TARGET)
$(TARGET): main.c
    $(CC) $(CFLAGS) main.c -o $(TARGET)
clean:
    rm -f $(TARGET)
amrutha@blue:~/skill_week_12$ make clean && make
rm -f fileio
gcc -Wall -Wextra main.c -o fileio
amrutha@blue:~/skill_week_12$ ls -l
-rwxr-xr-x 1 amrutha amrutha 16800 fileio
amrutha@blue:~/skill_week_12$
```

Observation

The Makefile uses gcc with -Wall and -Wextra. Its recipe lines are indented with tabs in the actual Makefile; the terminal panel shows the intended indentation as spaces for readability.

Result

The build process and basic deliverable structure were reviewed. A release tag should only be added after final files are committed and verified.

Task 4: Demonstration and Final Review

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

Program

The final demonstration combines the pipeline run with shell job monitoring. It is intended to show how commands exchange data through a pipe and how background jobs are listed.

Input

Commands entered: sleep 60 &, sleep 90 &, jobs -l, and ./fileio.

Output

```
amrutha@blue:~/skill_week_12$ sleep 60 &
[1] 1656
amrutha@blue:~/skill_week_12$ sleep 90 &
[2] 1666
amrutha@blue:~/skill_week_12$ jobs -l
[1]- 1656 Running                 sleep 60 &
[2]+ 1666 Running                 sleep 90 &
amrutha@blue:~/skill_week_12$ ./fileio
Pipeline: ls -l | wc -l
Number of output lines: 7
Pipeline completed successfully.
amrutha@blue:~/skill_week_12$
```

Observation

The terminal panel illustrates two running background jobs and a pipeline run. Verify the process IDs and output against your own terminal before submission, because process IDs vary between runs.

Result

The demonstration workflow covers pipeline execution and job monitoring.